

3.1 Thermometer

Thermometer

Overview

In this tutorial, you will use an OLED display to show the temperature.

Component Required:

- (1) x Elegoo ESP32
- (1) x OLED Display
- (1) x 10k ohm resistor
- (1) x Thermistor
- (2) x 400 tie-points Breadboard
- (6) x M-M wires (Male to Male jumper wires)



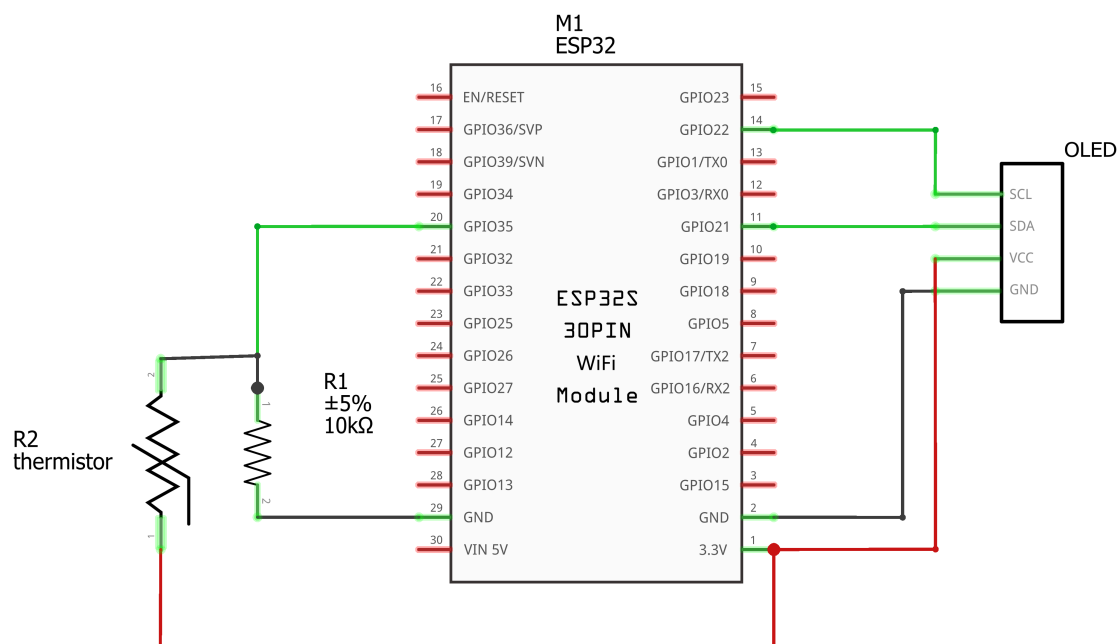
Component Introduction

Thermistor

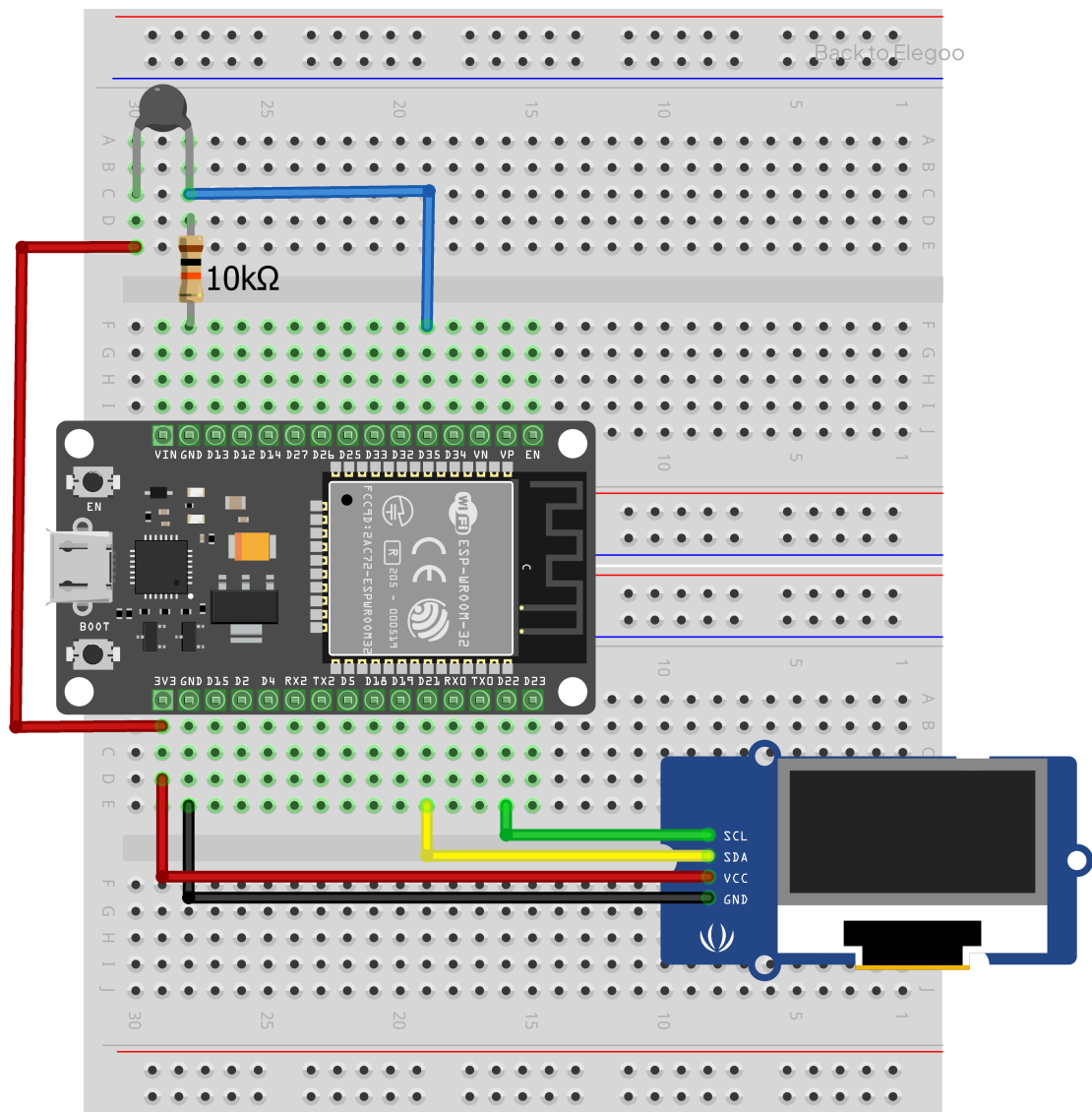
A thermistor is a thermal resistor - a resistor that changes its resistance with temperature. Technically, all resistors are thermistors - their resistance changes slightly with temperature - but the change is usually very small and difficult to measure. Thermistors are made so that the resistance changes drastically with temperature. It can be 100 Ohms or more, per degree of change.

There are two kinds of thermistors, NTC (negative temperature coefficient) and PTC (positive temperature coefficient). In general, you will see NTC sensors used for temperature measurement. PTC's are often used as resettable fuses - an increase in temperature increases the resistance which means that as more current passes through them, they heat up and 'choke back' the current, quite handy for protecting circuits!

Connection Schematic



Wiring diagram



Code

You can click the blue text link to download the program file to your local device, and double-click the file to open it after the download is complete. Please note: Before opening the file, ensure that you have installed the Arduino IDE development environment and completed the installation of relevant components such as the board support package and driver corresponding to the ESP32 development board. If you have any questions about this operation process, you can refer to the "part 1" chapter of the document for detailed guidance.

 [DOWENLOAD thermometer.ino](#)

Before you can run this, make sure that you have installed the and libraries or re-install them, if necessary. Otherwise, your code won't work.

With this code, the ESP32 reads temperature from an NTC thermistor, applies noise filtering, calculates temperature using the Steinhart-Hart equation, and displays it on an OLED screen. The temperature is updated every second.

Click the Serial Monitor button to turn on the serial monitor. The basics about the serial monitor are introduced in details in tutorial 4 in part 2.

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
```

Library Includes:

- ▶ [#include <Wire.h>](#) : ESP32 built-in library for I2C communication, used to communicate with the OLED display.
- ▶ [#include <Adafruit_GFX.h>](#) : Adafruit Graphics Library, provides basic graphics drawing functions for the OLED display.
- ▶ [#include <Adafruit_SSD1306.h>](#) : Adafruit SSD1306 OLED driver library, used to control the OLED display.

```
int tempPin = 35; // ADC input pin for NTC thermistor (ESP32 ADC1 channel)
```

ADC Pin Definition: `int tempPin = 35;`[Back to Elegoo](#)

Defines the ADC input pin for the NTC thermistor, using ESP32's ADC1 channel (GPIO 35).

```
// OLED display configuration
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1 // Reset pin not used with ESP32
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);
```

OLED Display Configuration:

- ▶ `#define SCREEN_WIDTH 128` : Defines the OLED display width as 128 pixels.
- ▶ `#define SCREEN_HEIGHT 64` : Defines the OLED display height as 64 pixels.
- ▶ `#define OLED_RESET -1` : Defines the OLED reset pin, not used with ESP32 so set to -1.
- ▶ `Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);` : Creates an OLED display object with the specified parameters.

```
// Series resistor used in the voltage divider with the NTC thermistor
const float seriesResistor = 10000.0; // 10kΩ
```

Series Resistor Definition: `const float seriesResistor = 10000.0;`

Defines the value of the series resistor used in the voltage divider circuit with the NTC thermistor (10kΩ).

```
void setup() {
  Serial.begin(115200);

  // Initialize I2C with SDA=21 and SCL=22
  Wire.begin(21, 22);

  // Initialize the OLED display
  if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println(F("SSD1306 allocation failed"));
    while (1);
  }

  delay(2000);
  display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(WHITE);
  display.setCursor(15, 0);
}
```

Setup Function:

- ▶ `Serial.begin(115200);` : Initializes serial communication at 115200 baud rate for debugging.
- ▶ `Wire.begin(21, 22);` : Initializes I2C communication with SDA=21 and SCL=22.
- ▶ `if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) { ... }` : Initializes the OLED display with internal charge pump and I2C address 0x3C. If initialization fails, prints error message and enters infinite loop.
- ▶ `delay(2000);` : Delays 2 seconds to allow OLED initialization.
- ▶ `display.clearDisplay();` : Clears the OLED display content.
- ▶ `display.setTextSize(1);` : Sets text size to 1.
- ▶ `display.setTextColor(WHITE);` : Sets text color to white.
- ▶ `display.setCursor(15, 0);` : Sets cursor position to (15, 0).

```
float calculateTemperature() {

  // --- 1. Take multiple ADC samples for noise reduction ---
  int tempReading = 0;
  for (int i = 0; i < 5; i++) {
    int val = analogRead(tempPin);

    1 | // Constrain ADC reading to avoid extreme outliers (0 or 4095)
    2 | tempReading += constrain(val, 10, 4085);
```

```
3 | delay(3);
```

[Back to Elegoo](#)

```
}  
tempReading /= 5; // Average value
```

Temperature Calculation Function (Part 1):

- `float calculateTemperature()` : Function to read thermistor value, apply noise filtering, and calculate temperature.
- `int tempReading = 0;` : Initializes temperature reading variable.
- `for (int i = 0; i < 5; i++) { ... }` : Takes 5 ADC samples for noise reduction.
- `int val = analogRead(tempPin);` : Reads ADC input value.
- `tempReading += constrain(val, 10, 4085);` : Constrains ADC reading to avoid extreme values, then adds to tempReading.
- `delay(3);` : Delays 3 milliseconds between samples to avoid reading too quickly.
- `tempReading /= 5;` : Calculates average of 5 readings to reduce noise.

```
// --- 2. Convert ADC reading to thermistor resistance ---  
// ESP32 uses 12-bit ADC (0-4095 range)  
float ratio = (4095.0 / tempReading) - 1.0;  
float ntcResistance = seriesResistor * ratio;  
  
// Safety check: prevent log(0) or log of negative values  
if (ntcResistance <= 0) {  
    return -1.0; // Error flag  
}
```

Temperature Calculation Function (Part 2):

- `float ratio = (4095.0 / tempReading) - 1.0;` : Calculates the ratio between ADC reading and reference value.
- `float ntcResistance = seriesResistor * ratio;` : Calculates NTC thermistor resistance based on voltage divider principle.
- `if (ntcResistance <= 0) { return -1.0; }` : Safety check to prevent log(0) or log of negative values, returns -1.0 as error flag.

```
// --- 3. Apply the Steinhart-Hart thermistor equation ---  
double lnR = log(ntcResistance);  
double tempK = 1.0 / (  
    0.001129148 +  
    (0.000234125 + 0.0000000876741 * lnR * lnR) * lnR  
);  
  
float tempC = tempK - 273.15; // Convert Kelvin → Celsius  
  
// Limit temperature to a realistic range (-10°C to 80°C)  
return constrain(tempC, -10.0, 80.0);  
}
```

Temperature Calculation Function (Part 3):

- `double lnR = log(ntcResistance);` : Calculates natural logarithm of thermistor resistance.
- `double tempK = 1.0 / (0.001129148 + (0.000234125 + 0.0000000876741 * lnR * lnR) * lnR);` : Calculates temperature in Kelvin using Steinhart-Hart equation.
- `float tempC = tempK - 273.15;` : Converts Kelvin temperature to Celsius.
- `return constrain(tempC, -10.0, 80.0);` : Limits temperature to realistic range (-10°C to 80°C) and returns calculated temperature.

```
void loop() {  
    float tempC = calculateTemperature();  
  
    display.clearDisplay();  
    display.setTextSize(2);  
    display.setCursor(0, 3);  
    display.print("Temp:");  
  
    display.setCursor(30, 25);  
  
    // Display error message if the reading was invalid  
    if (tempC == -1.0) {
```

```
display.print("Err");
} else {
display.print(tempC, 1);
display.print("C");
}

display.display();
delay(1000);
}
```

[Back to Elegoo](#)

Loop Function:

- `float tempC = calculateTemperature();` : Calls `calculateTemperature()` function to read and calculate temperature.
- `display.clearDisplay();` : Clears OLED display content.
- `display.setTextSize(2);` : Sets text size to 2 for clearer temperature display.
- `display.setCursor(0, 3);` : Sets cursor position to (0, 3).
- `display.print("Temp:");` : Prints "Temp:" text on OLED display.
- `display.setCursor(30, 25);` : Sets cursor position to (30, 25) for temperature value.
- `if (tempC == -1.0){ display.print("Err"); }` : Displays "Err" if temperature reading is invalid (return value is -1.0).
- `else { display.print(tempC, 1); display.print("C"); }` : Displays temperature value (with 1 decimal place) and "C" unit if reading is valid.
- `display.display();` : Displays buffer content on OLED display.
- `delay(1000);` : Delays 1 second before repeating the loop.

Troubleshooting

- **OLED Not Displaying:** Check I2C connections and ensure display address is 0x3C
- **Temperature Reading Error:** Verify thermistor and resistor connections
- **Incorrect Temperature:** Ensure you're using the correct Steinhart-Hart coefficients for your NTC thermistor
- **Serial Monitor Issues:** Set baud rate to 115200

[← Overview](#)

[74HC595 and segment display→](#)